COMPUTATIONAL INTELLIGENCE

Project Proposal Brief

1. Student(s) Information:

Name: Zainab Zahid	**ID:** zz08969	**Batch:** 2027
Name: Hazeera Hashim	**ID:** hm09247	**Batch:** 2027
Name: Minhaj ul Hassan	**ID:** mu08984	**Batch:** 2027

2.1. Problem Statement

Title: Optimizing a Chess Opening Repertoire using a Co-evolutionary Genetic Algorithm

In chess, club-level players rated 1600-1800 Elo must prepare a limited set of opening lines to study before competitive games. This set is called an *opening repertoire.* Since hundreds of opening variations exist, players must carefully choose a small number of lines that are both effective and realistic to memorize.

This project addresses the following optimization problem:

> From a pool of 300-500 candidate opening lines, select 30-50 lines that maximize expected performance in real games while keeping the total memorization effort manageable.

This is a constrained *combinatorial optimization problem*. For example, selecting 38 lines out of 400 possible options results in billions of possible combinations, making exhaustive search computationally infeasible.

The problem is difficult because simply choosing the lines with the highest win rates does not produce a strong repertoire. High-performing lines may overlap, leave gaps against certain opponent first moves, or perform poorly against specific playing styles.

Therefore, the selected repertoire must satisfy the following constraints:

1. Size constraint: 30-50 total lines.
2. Color balance: At least 12 lines for White and 12 for Black.
3. Coverage: At least one response to each major first move (1.e4, 1.d4, 1.c4, 1.Nf3).
4. Complexity: Total number of moves across all selected lines must remain under a memorization limit.

Additionally, performance should be evaluated against realistic and varied opponent behavior rather than a fixed average. The goal is to build a repertoire that is robust, balanced, and practical for real-world competition.

To solve this problem, we propose a ***Co-evolutionary Genetic Algorithm*****,** where one population represents candidate repertoires and a second population represents adaptive opponent models. Both populations evolve simultaneously, leading to a repertoire that is optimized for both strength and robustness.

2.2. Motivation

Chess is a very popular strategy game, but amateur players (1600-1800 Elo) do not have a clear method for choosing the best openings to study. Most club players spend a lot of time learning openings, but their process is often unorganized and inefficient.

Currently, players usually choose openings in three ways:

1. **Manual selection**: using books, coaches, or online content. These are often subjective and designed for grandmasters, not amateurs with limited time.
2. **Engine analysis**: tools like Stockfish suggest the best moves, but they do not tell players which openings to study or how to create a practical set.
3. **Win-rate selection**: picking openings with the highest historical success. This ignores coverage gaps, overlapping lines, and weaknesses against certain playing styles.

In reality, opponents are not average. Some are aggressive, some are defensive. A repertoire that works on average may fail against specific styles. This is why we use a Co-evolutionary Genetic Algorithm, which evolves both the repertoire and opponent models. This helps create a balanced, strong, and practical set of openings that works well against different types of opponents.

2.3. Relevant Research.

Aksenov, P. (2004). Genetic algorithms for optimising chess position scoring (Master's thesis, University of Joensuu, Department of Computer Science).

David, E. (O.), van den Herik, H. J., Koppel, M., & Netanyahu, N. S. (2014). Genetic algorithms for evolving computer chess programs. IEEE Transactions on Evolutionary Computation, 18(5), 779–789. https://doi.org/10.1109/TEVC.2014.2306978

De Marzo, G., & Servedio, V. D. P. (2023). Quantifying the complexity and similarity of chess openings using online chess community data. Scientific Reports, 13(1), 5327. https://doi.org/10.1038/s41598-023-31658-w

Fogel, D. B. (2004). A self-learning evolutionary chess program. Proceedings of the IEEE, 92(12), 1947–1964. https://doi.org/10.1109/JPROC.2004.837423

Lassabe, N., Sanchez, S., Luga, H., & Duthen, Y. (2007). Genetically programmed strategies for chess endgame. In Proceedings of the 2007 IEEE Symposium on Computational Intelligence and Games (pp. 831–838). IEEE. ISBN 1-59593-186-4

Ramos, N., Salgado, S., & Rosa, A. C. (2016). Chess Player by Co-Evolutionary Algorithm. arXiv preprint arXiv:1605.06710. https://arxiv.org/abs/1605.06710

2.4. Dataset

The primary data source is the *Lichess Opening Explorer API*, a free, publicly available interface aggregating hundreds of millions of real human games, queryable by rating range and time control.

Data is collected by systematically querying the API across commonly played positions, filtered to 1600-1800 Elo rapid and classical games. Each opening line in the pool records its full move sequence, win rate, draw rate, total games played, color assignment, and opponent response frequency distribution. Positions with insufficient game counts are excluded to ensure statistical reliability. The resulting dataset contains approximately 300-500 distinct opening lines, serving as the static input to all optimization runs.

2.5. Simulator

Once the best repertoire is produced by the GA, it is validated by simulating games using the *python-chess* library. The repertoire is loaded as an opening book, for the first 8-10 moves, whenever a position is reached that the repertoire covers, the matching move is played automatically. The opponent plays moves weighted by real Lichess frequency data, meaning common responses appear more often than rare ones, approximating a realistic 1600-1800 player. After the repertoire's prepared moves are exhausted, both sides continue until the game ends naturally. Results across 500 simulated games are compared against three baselines, a greedy top win-rate selection, a random selection, and a human-curated repertoire, using win rate, draw rate, and overall performance score.

2.6. Glossary

Coverage: Whether the repertoire contains at least one prepared response to each major first move an opponent might play (1.e4, 1.d4, 1.c4, 1.Nf3). A repertoire with gaps in coverage leaves the player unprepared against certain opponent choices.

Elo: The standard numerical rating system used in chess to measure player strength. A rating of 1600-1800 corresponds to a serious club-level amateur player. Grandmasters typically rate above 2500.

Memorization limit: A cap on the total number of moves across all selected lines in a repertoire. Since each line is 6–12 moves long and a repertoire contains 30–50 lines, the total moves to memorize could range from 200 to 600. The memorization limit ensures the optimized repertoire remains practically learnable for a human player with limited study time.

Opening book: A pre-compiled lookup table of opening moves used during game simulation. When the simulator reaches a position covered by the repertoire, it consults the opening book and plays the prepared move automatically rather than calculating from scratch.

Opening line: A specific sequence of moves played at the start of a chess game, typically 6–12 moves long. For example, the sequence 1.e4 e5 2.Nf3 Nc6 3.Bb5 is a well-known opening line called the Ruy Lopez. Each line represents a prepared path a player has studied and memorized.

Opponent model: A probability distribution representing a specific playing style, encoding how likely a player of that style is to respond with each available move at major decision points in the game.

Opening repertoire: The complete collection of opening lines a player has prepared for competitive play, covering responses for both White and Black across different opponent first moves.

python-chess: A free, open-source Python library for chess move generation, validation, and game simulation. Used in this project to run simulated games for validating the final repertoire.